University of the Bundeswehr Munich

Institute for **Autonomous Driving**

Modeling Robotics Dataset Construction as an Artifact-Based Build Process

Leon Pohl, Lukas Beer, George Sebastian and Mirko Maehlisch |

IEEE CASE 2026 | August 20, 2026 | ThBT5.4 | Room 205 | 4:20-4:40 pm |

2026 IEEE 22nd International Conference on Automation Science and Engineering (CASE)

IEEE Advancing Technology for Humanity

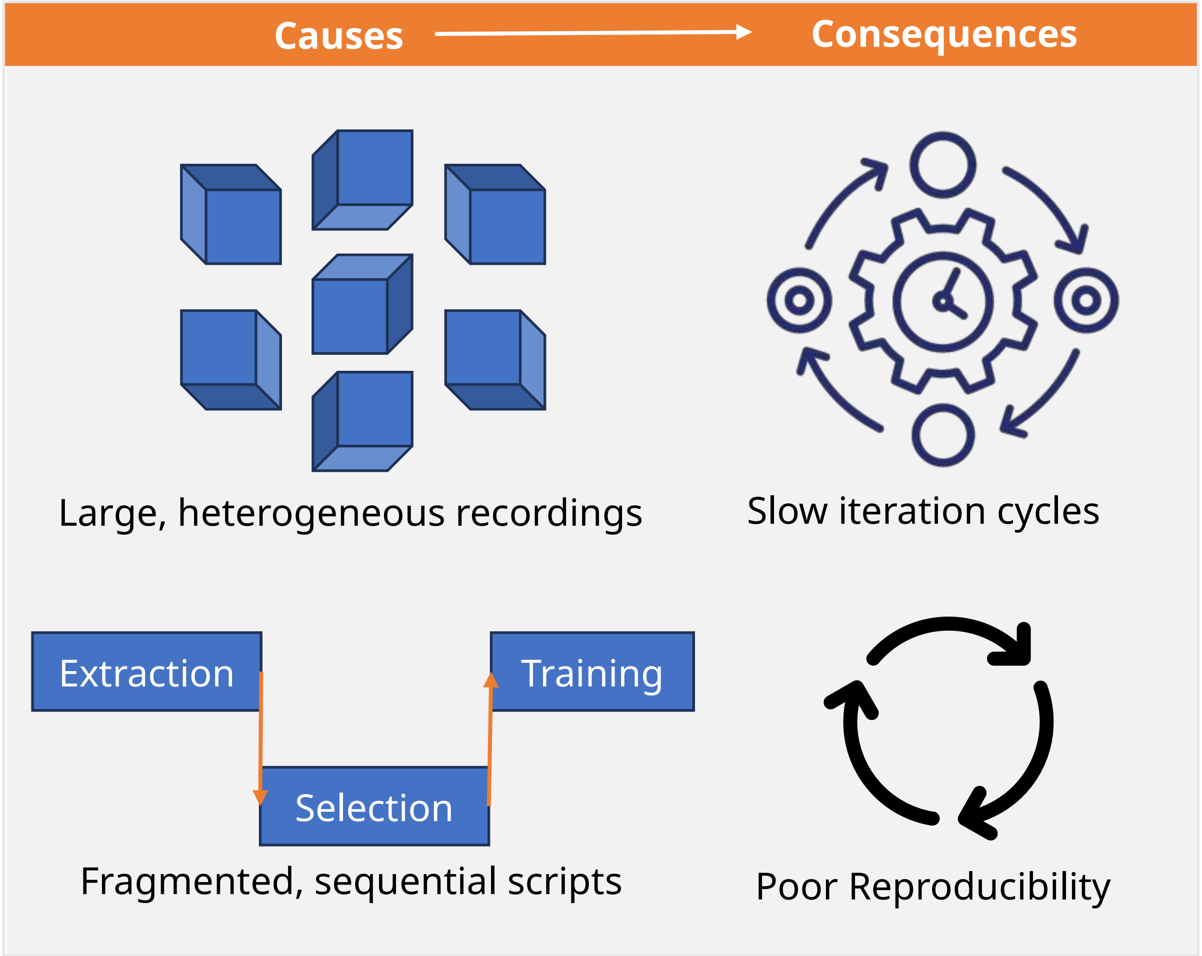
Motivation – Large Scale Robotic Data

Credits: Anton Backhaus

Recording robot data is easy. Turning every update into a training-ready dataset is the hard part.

Dataset Preparation Is Slow and Hard to Reproduce

- Dataset preparation often consists of separate, sequential scripts.
- Dependencies and intermediate results are not tracked explicitly.
- A small input change can therefore trigger unnecessary work across the entire pipeline.

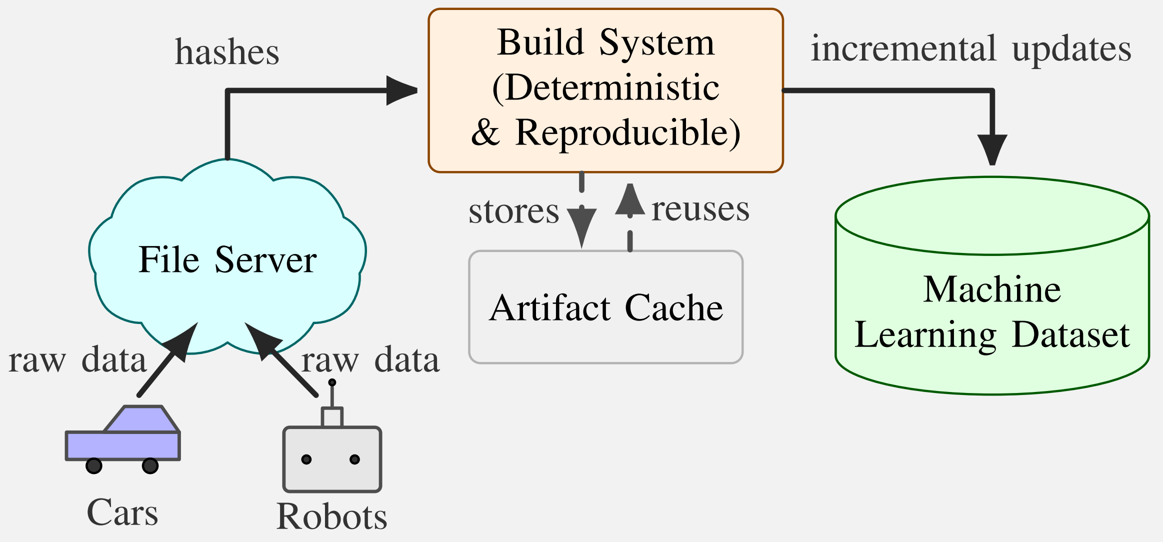


The pipeline must know what changed, reuse what remains valid, and rebuild only what is affected.

Key Idea: Treat Dataset Construction as a Build Process

- Recordings become source artifacts.
- Hashes reveal what changed.
- Cached results are reused.

Dataset Construction with Build System

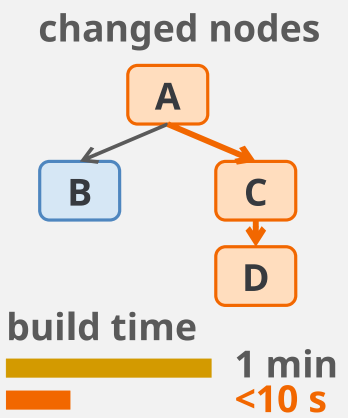


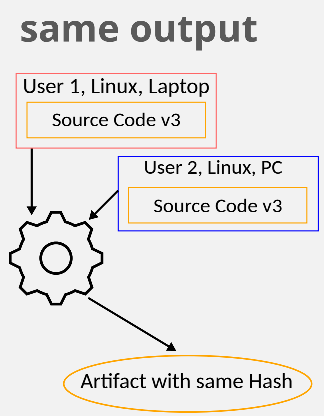
Relevant Build-System Properties

Incremental + Deterministic Builds

Incremental:
only changed artifacts rebuild.

Deterministic:
same inputs produce the same output.



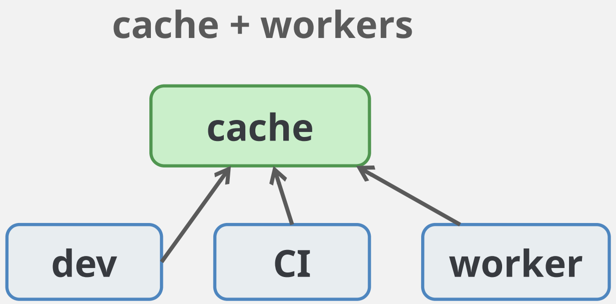


saves time; higher reliability.

Remote Cache + Execution

Remote cache:
share artifacts across developers and CI.

Remote execution:
run independent actions on workers.

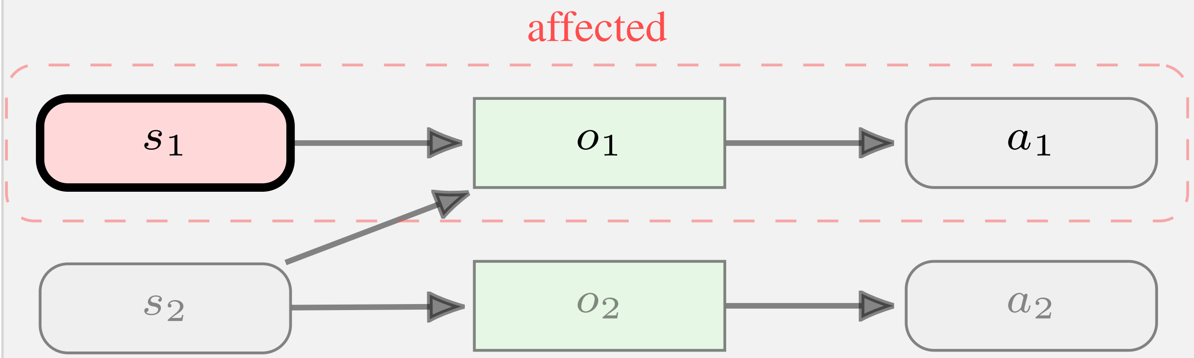


Reuse; scale compute.

Methodology — Artifact-Based Dataset Construction

Selective Recomputation (Bagzel)

Source artifact → Operation → Output Artifact

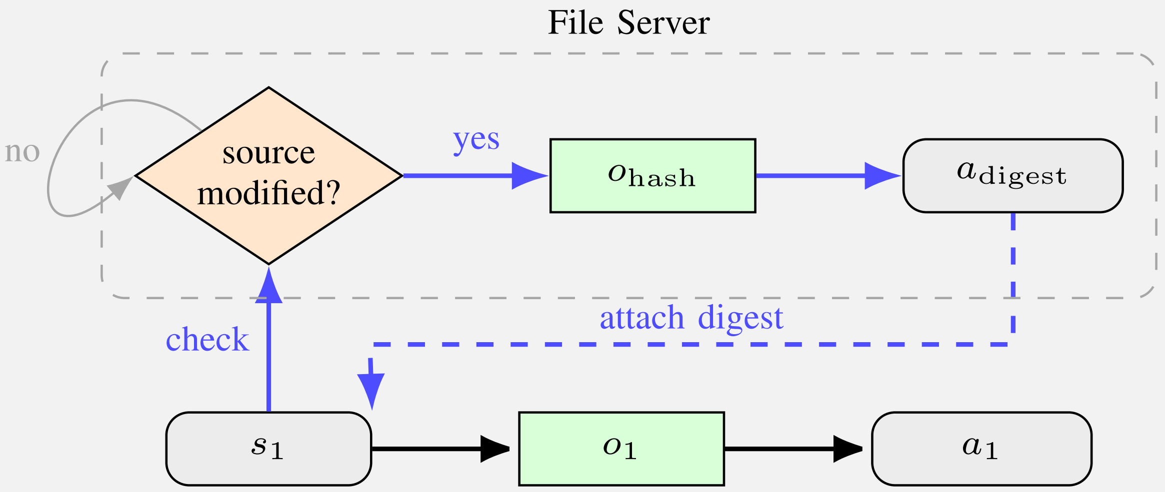


Action digest = hash of inputs, operation, and parameters

- **New digest:** rebuild the affected branch.
- **Same digest:** reuse cached artifacts.

Server-Side Digest Management (Bagzel-xattr)

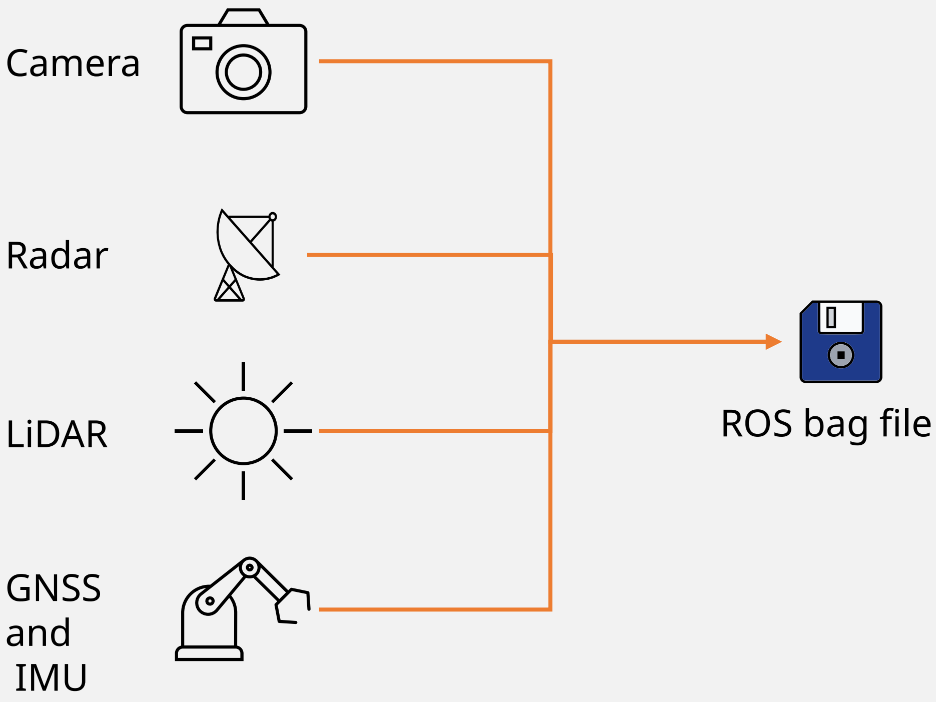
Bottleneck: Every build rehashes multi-GB sources.



- **Modified:** compute and store a new digest.
- **Unchanged:** reuse the server-stored digest

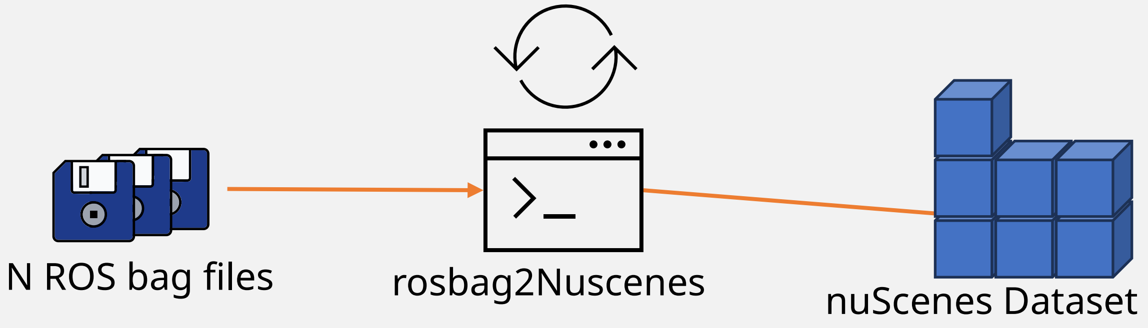
Recompute only dependent operations; hash large sources only when they change.

ROS Bag File



» Think of it as a “flight recorder” for robots
- all sensor and state data in one file.

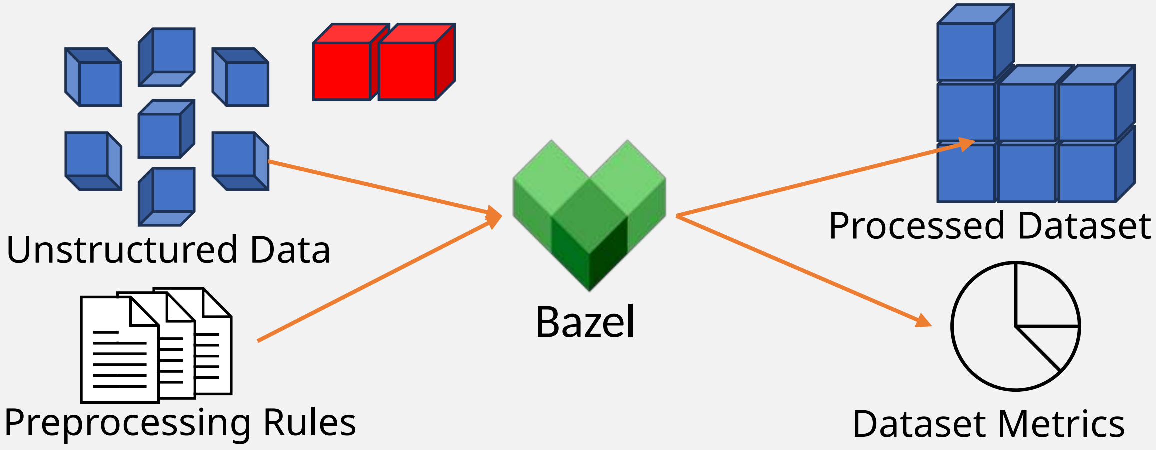
Sequential rosbag2Nuscenes baseline [24]



- Repeat (N times) - sequentially

Workflow adapted from [23].

Bagzel Dataset Preparation



Experimental Setup

Research Questions

RQ1: execution efficiency against rosbag2nuscenes.

RQ2: scaling from 5.1 GB to 20.4 GB.

RQ3: input granularity and server-side digests.

Build Modes

Cold: no reusable cache entries.

Warm: unchanged inputs and reusable artifacts.

Incremental: localized metadata change in one source artifact.

Workstation

- Intel i7-11700K (8c/16t), 64 GB RAM, 4 TB Samsung 990 PRO NVMe, Ubuntu 20.04.6, Bazel 8.3.1. No GPU Acceleration.

ROS 1 bag file (2.5 GB) $\xrightarrow{\text{converted}}$ ROS 2 bag file (2.6 GB)

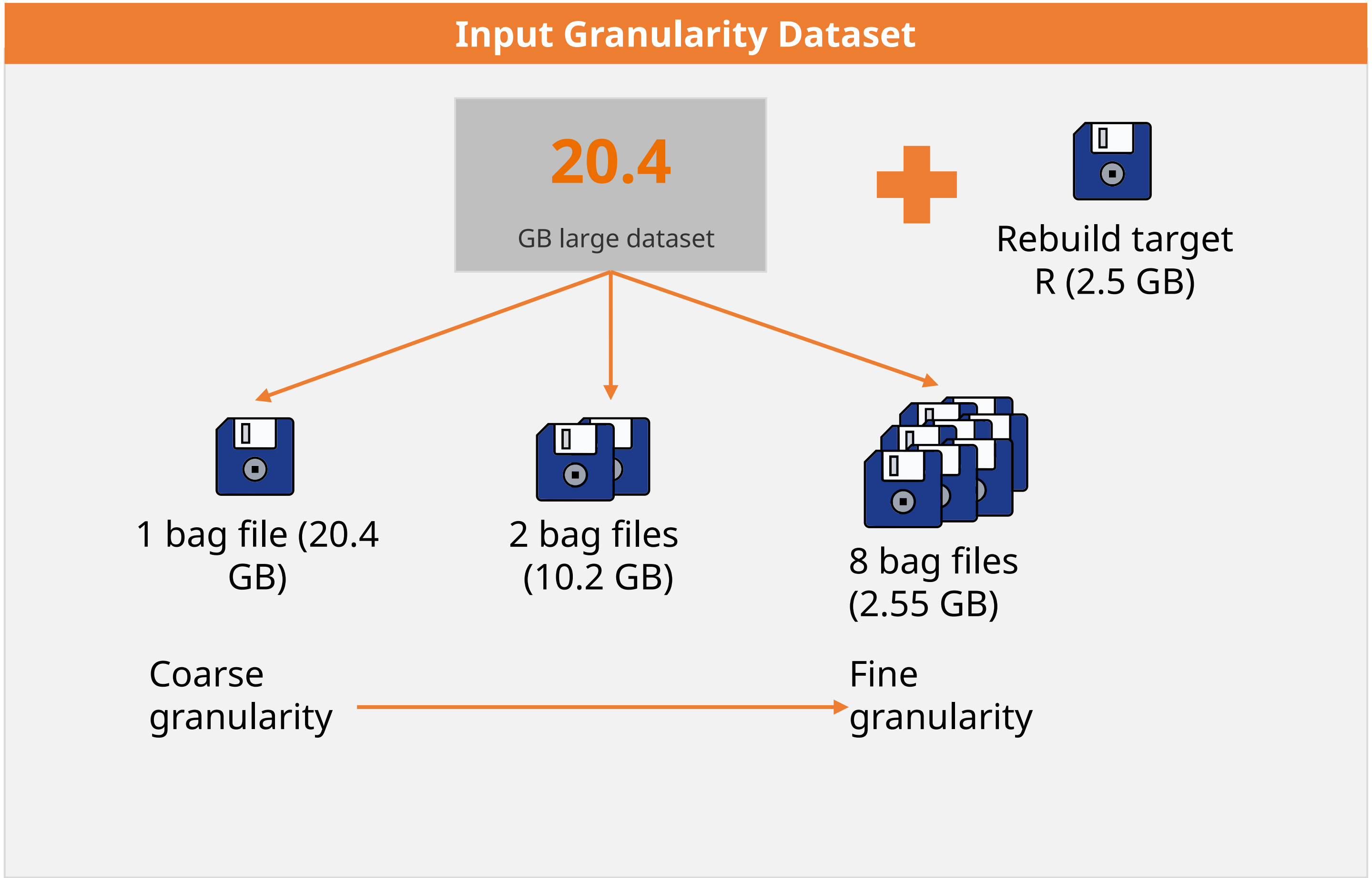
5.1 GB small dataset $\xrightarrow{2x}$ **10.2** GB medium dataset $\xrightarrow{2x}$ **20.4** GB large dataset

3x
median of independent runs

All experiments ran locally on the same workstation, with remote caching disabled.

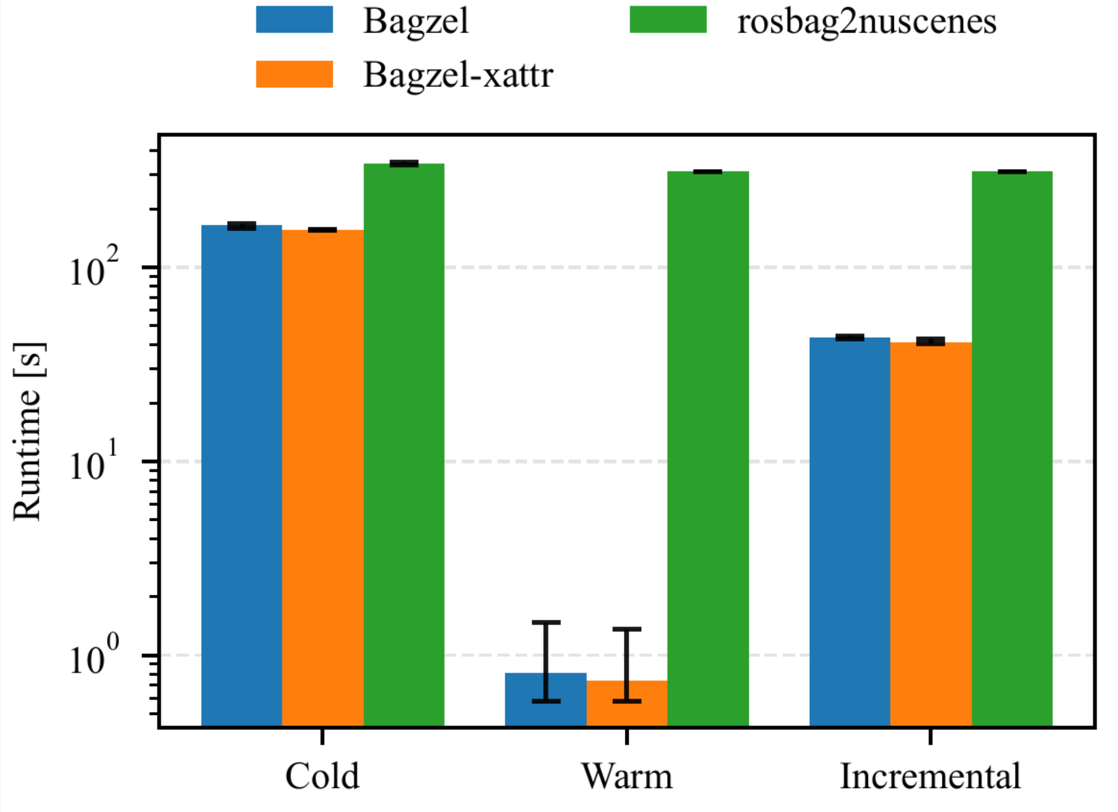
IEEE

Experimental Setup – Input Granularity Dataset



RQ1 – The Largest Speedups Occur in Iterative Builds

Runtime comparison on the large dataset (20.4 GB)



2.06x
cold build speedup

386x
warm build speedup

7.21x
incremental speedup

Key Numbers

Warm: 312.87 s -> 0.81 s.

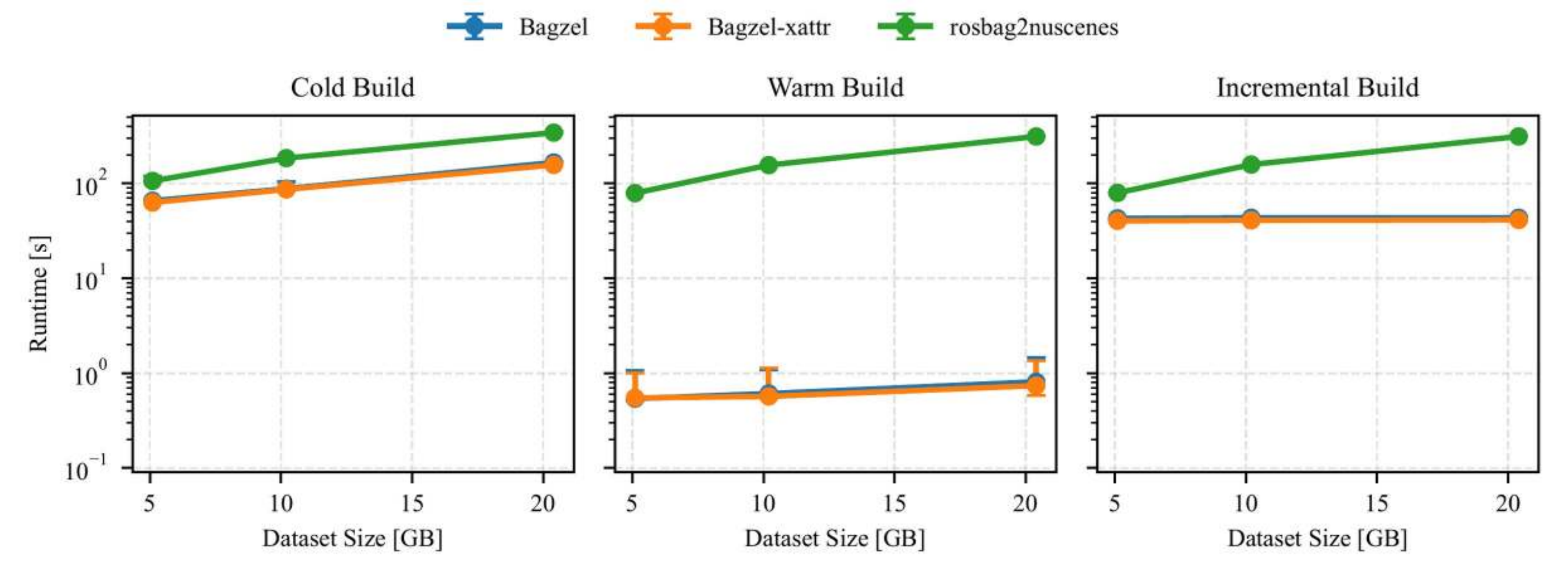
Incremental: 313.11 s -> 43.45 s.

Bagzel-xattr adds a further 4.9% incremental reduction.

The largest gains appear during warm and incremental builds.

RQ2 - Cached and Incremental Modes Scale Almost Flat

Runtime scaling from 5.1 GB to 20.4 GB



Linear-fit Slopes

Cold: Bagzel 6.74 vs baseline 15.50 s/GB.

Warm: Bagzel ~0.02 vs baseline 15.29 s/GB.

Incremental: Bagzel ~0.03 vs baseline 15.25 s/GB.

Interpretation

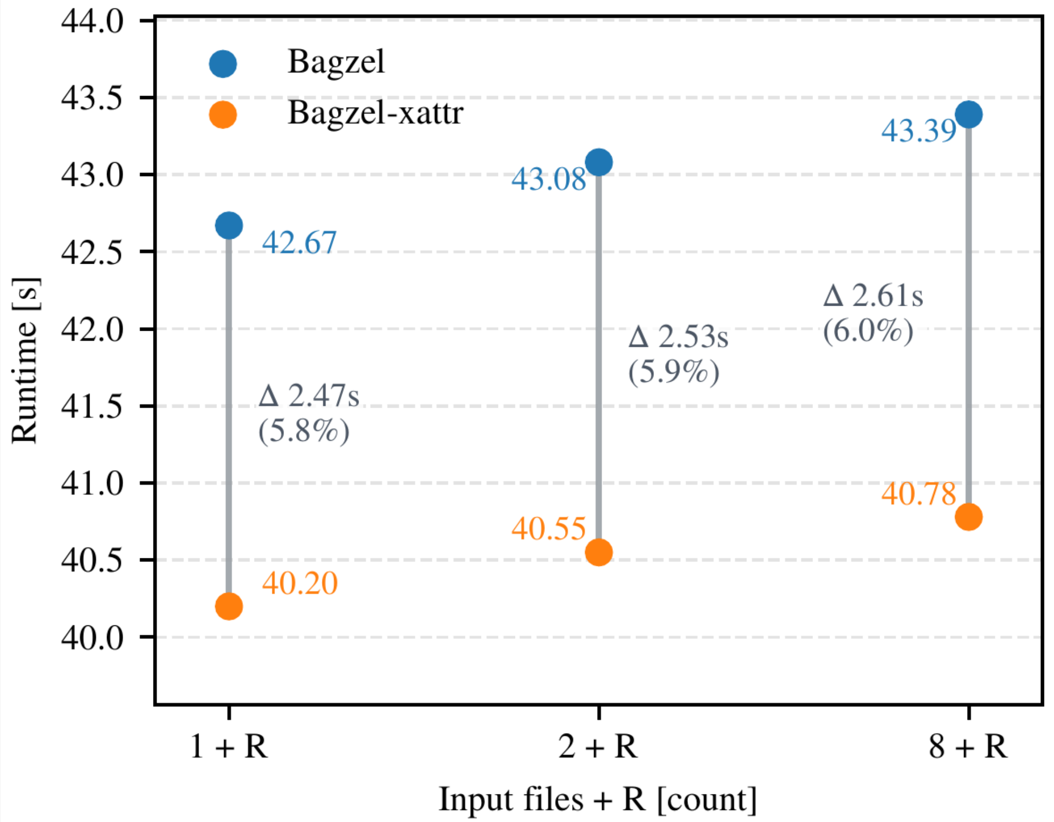
Sequential conversion grows with total dataset size.

Artifact reuse show only weak dependence on warm and local rebuild time.

Within the evaluated range, warm and incremental Bagzel builds show only weak dependence on dataset size.

RQ3 - Server-side Digests Give Consistent Extra Gains

Incremental runtime under fixed total data volume



5.9%
mean runtime reduction vs Bagzel

2.53s
mean absolute gain on this workstation

Takeaway

Input granularity has only a minor effect in this range.

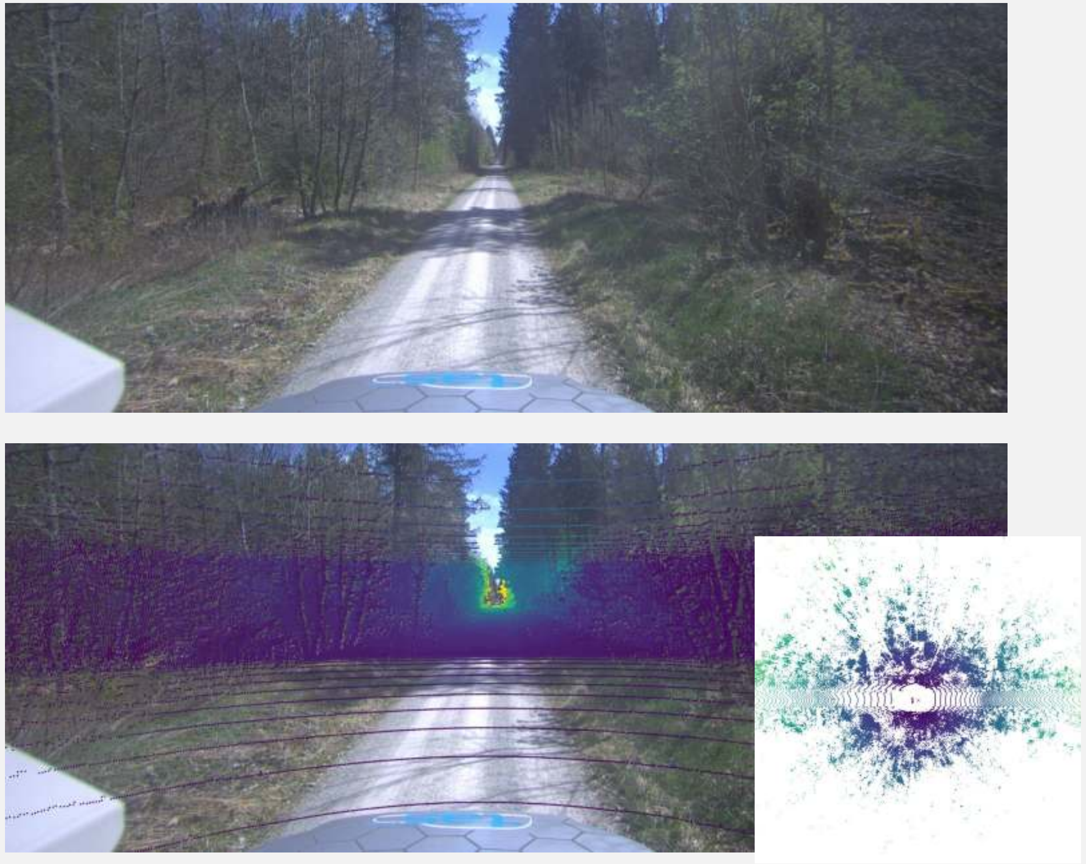
Bagzel-xattr is faster for one, two, and eight-file conditions.

The advantage should matter more with larger inputs or more artifacts.

Server-side digest management gives modest but stable gains.

Conclusion and Outlook

Bagzel Output Example



Key Findings

Deterministic actions and explicit dependencies are designed to support reproducible dataset generation.

Experimentally, Bagzel achieved up to 386.26× faster warm builds and 7.21× faster incremental builds.

Warm and incremental runtimes scaled weakly with dataset size; Bagzel-xattr added a 5.9% average improvement.

Limitations and Next Steps

Runtime-focused evaluation on one workstation.

Datasets span 5.1 GB to 20.4 GB.

Next: larger datasets, distributed execution, and remote caches.

Build robotics datasets like software: iterate faster, rebuild less, and support reproducibility by design.

References

- [23] J. Chrosniak and M. Behl, “ROSBag2NuScenes: Share the Bags, Spread the Joy—Autonomous Vehicle ROS Datasets Deploy,” presented at ROSCon 2023, New Orleans, LA, USA, Oct. 2023. [Online]. Available: ROSCon presentation slides
- [24] A. Kulkarni, J. Chrosniak, E. Ducote, F. Sauerbeck, A. Saba, U. Chirimar, J. Link, M. Cellina, and M. Behl, “RACECAR—The dataset for high-speed autonomous racing,” in *Proc. 2023 IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Detroit, MI, USA, Oct. 2023, pp. 11458–11463, doi: 10.1109/IROS55552.2023.10342053.

Dive Deeper: Paper, Code, and Related Talks

Scan the paper, implementation, or related talk resources.

Paper

arXiv:2606.00162

ArXiv paper and citation.

arxiv.org/abs/2606.00162

Source Code

Bagzel repository

Implementation, examples, and BUILD rules.

github.com/UniBwTAS/bagzel

Build-System Talk

BazelCon 2025

Same topic for build-system engineers.

youtu.be/xa-4_hdk_C4

Robotics Talk

ROSCon FR & DE 2025

Same topic for robotics workflows.

vimeo.com/showcase/12079514

For follow-up: cite the paper, reproduce the implementation, or use the talks for deeper context.

Questions?

Modeling Robotics Dataset Construction as an Artifact-Based Build Process

IEEE CASE 2026 | ThBT5.4 | Room 205 | 4:20-4:40 pm

Thanks to Lukas Beer, George Sebastian, and Mirko Maehlisch.